

### **Primary Use Cases:**

- **Create Account**
- **Login**
- **Logout**
- **Create or Join Pond**
- **Complete Daily Check-In**
- **View Pond**
- **Send Ribbit**
- **Acknowledge Ribbit**
- **Answer Daily Discussion Prompt**
- **Request Smart Connection Assistant Suggestion**

**Use case:** CreateAccount

**ID:** UC-UA-01

**Actors:** PondMate/User

#### **Preconditions:**

- User has opened PondLite.
- User does not already have an account, or wants to create a new one.

### **Flow of events (Main Success Scenario):**

1. User selects Create Account.
2. System displays the account creation form.
3. User enters required account information.
4. User submits the form.
5. System validates the submitted information.
6. System encrypts the user's password.
7. System creates and saves the new account.
8. System confirms account creation.
9. System redirects user to login, or automatically logs the user in.

### **Alternate flows:**

#### **A1 — Invalid or missing information**

- **A1.1** At Step 5, system detects missing or invalid information.
- **A1.2** System displays an error message.
- **A1.3** User corrects the information and resubmits.
- **A1.4** Use case resumes at Step 5.

#### **A2 — Account already exists**

- **A2.1** At Step 5, system detects that the email/username is already associated with an account.
- **A2.2** System displays an account already exists message.
- **A2.3** User may choose to log in instead.

**Postconditions:**

- New user account is created and saved.
- User can now log in to PondLite.

**Use case:** Login

**ID:** UC-UA-02

**Actors:** PondMate/User

**Preconditions:**

- User has an existing account.
- User is not currently logged in.

**Flow of events (Main Success Scenario):**

1. User opens PondLite or selects Login.
2. System displays the login form.
3. User enters username/email and password.
4. User submits the login form.
5. System validates the submitted credentials.
6. System generates an authentication token/session.
7. System stores the active token/session.
8. System redirects user to the Pond/Home page.

**Alternate flows:**

**A1 — Invalid credentials**

- **A1.1** At Step 5, system detects that the username/email or password is incorrect.
- **A1.2** System displays an invalid login message.
- **A1.3** User may re-enter credentials and submit again.
- **A1.4** Use case resumes at Step 5.

**A2 — Account does not exist**

- **A2.1** At Step 5, system finds no account matching the submitted username/email.
- **A2.2** System displays an account not found message.
- **A2.3** User may choose to create an account.

**Postconditions:**

- User is logged in.
- User has an active token/session.
- User can access protected PondLite features.

**Use case:** Logout

**ID:** UC-UA-03

**Actors:** PondMate/User

**Preconditions:**

- User has an existing account.
- User is currently logged in.
- User has an active token/session.

**Flow of events (Main Success Scenario):**

1. User selects Logout.
2. System receives the logout request.
3. System invalidates or deletes the active token/session.
4. System clears the token/session from the client side.
5. System redirects user to the Login page.
6. System displays confirmation that the user has logged out.

**Alternate flows:**

**A1 — Token/session already expired**

- **A1.1** At Step 3, system detects that the token/session is already expired.
- **A1.2** System clears the token/session from the client side.
- **A1.3** System redirects user to the Login page.

**Postconditions:**

- User is logged out.
- User no longer has an active token/session.
- Protected PondLite features require login again.

**Use case:** CompleteDailyCheckIn

**ID:** UC-PL-01

**Actors:** PondMate/User

**Preconditions:**

- User is logged in.
- User is connected to a Pond/Relationship.

- User has not completed a check-in for the current day within that Pond, or is allowed to edit it.

#### **Flow of events (Main Success Scenario):**

1. User enters the Pond page.
2. System checks if a daily check-in has been completed.
3. System prompts the user to complete their daily check-in.
4. System displays the emotion wheel.
5. User selects up to three emotions (primary required).
6. User optionally enters a message for their PondMate.
7. User submits the check-in.
8. System validates that the user belongs to the current Pond/Relationship.
9. System saves the check-in with the RelationshipId and UserAccountId.
10. System updates the user's frog state based on the selected emotions.
11. System updates the Pond view.
12. System makes the optional check-in message available only to the PondMate in the same Pond.

#### **Alternate flows:**

##### **A1 — User skips optional message**

- **A1.1** At Step 6, user chooses not to enter a message.
- **A1.2** System proceeds with emotion-only check-in.
- **A1.3** Use case continues at Step 7.

##### **A2 — User already checked in**

- **A2.1** At Step 2, system detects an existing check-in for the day.
- **A2.2** System either:
  - Prevents a second check-in and displays confirmation of completion, or
  - Allows the user to edit the existing check-in.
- **A2.3** If editing is allowed, flow resumes at Step 4.

#### **Postconditions:**

- User's daily emotional state is recorded for the current Pond/Relationship.
- Frog state reflects the user's emotions.
- Optional message is available only to the PondMate in the same Pond.
- System marks the user as checked in for the current day.

**Use case:** ViewPond

**ID:** UC-PL-02

**Actors:** PondMate/User

**Preconditions:**

- User is logged in.
- User has access to a Pond/Relationship space.
- User is a RelationshipMember of the Pond/Relationship being viewed.
- User has completed today's check-in, or the system allows limited Pond access before check-in.

**Flow of events (Main Success Scenario):**

1. User selects or enters the Pond page.
2. System verifies that the user has a valid session.
3. System verifies that the user is a member of the requested Pond/Relationship.
4. System retrieves only Pond data associated with that Relationship.
5. System retrieves both PondMates' frog states.
6. System retrieves today's check-in status for both PondMates.
7. System retrieves any active Ribbits, discussion prompts, or unread messages.
8. System displays the Pond screen.
9. System displays available Pond actions, such as check-in, reminders/ribbits, discussion, and AI suggestions.

**Alternate flows:****A1 — User has not completed daily check-in**

- **A1.1** At Step 5, system detects that the user has not checked in today.
- **A1.2** System prompts the user to complete the daily check-in.
- **A1.3** User completes the check-in.
- **A1.4** Use case resumes at Step 6.

**A2 — No PondMate connected**

- **A2.1** At Step 3, system detects that the user does not have a connected PondMate.
- **A2.2** System displays a message explaining that no PondMate is connected.
- **A2.3** System may provide an option to invite or connect with a PondMate.

**A3 — User is not authorized for Pond**

- **A3.1** At Step 3, system detects that the user is not a member of the requested Pond/Relationship.
- **A3.2** System denies access.
- **A3.3** System displays an unauthorized access message or redirects the user to their own Pond.

**Postconditions:**

- User sees the current Pond state.
- User can view frog status, emotional reflection, and available connection actions.

**Use case:** Send Ribbit

**ID:** UC-PL-03

**Actors:** PondMate/User

**Preconditions:**

- User is logged in.
- User is connected to a Pond/Relationship.
- User has a connected PondMate in the same Pond.
- User is viewing the Pond or Ribbit feature.

**Flow of events (Main Success Scenario):**

1. User selects Send Ribbit.
2. System displays the Ribbit creation form.
3. User chooses a Ribbit type, such as Thought, Gentle Request, Care Offer, or Task.
4. User enters a short message or selects a preset message.
5. User optionally selects a delivery time.
6. User submits the Ribbit.
7. System validates the Ribbit content.
8. System verifies that the sender and receiver belong to the same Pond/Relationship.
9. System saves the Ribbit with RelationshipId, SenderUserId, and ReceiverUserId.
10. System sends or schedules the Ribbit for the PondMate.
11. System confirms that the Ribbit was sent or scheduled.

**Alternate flows:**

**A1 — Missing or invalid content**

- **A1.1** At Step 7, system detects missing or invalid content.
- **A1.2** System displays an error message.
- **A1.3** User corrects the reminder/ribbit.
- **A1.4** Use case resumes at Step 6.

**A2 — User cancels**

- **A2.1** At any point before Step 6, user cancels the reminder/ribbit.
- **A2.2** System discards unsaved reminder/ribbit content.
- **A2.3** System returns user to the previous screen.

**Postconditions:**

- Ribbit is saved under the current Pond/Relationship.

- PondMate can receive, view, or acknowledge the Ribbit.
- Ribbit is not visible outside the shared Pond.

**Use case:** Acknowledge Ribbit

**ID:** UC-PL-04

**Actors:** PondMate/User

**Preconditions:**

- User is logged in.
- User has received a Ribbit from their PondMate.
- Ribbit belongs to the same Pond/Relationship as the user.
- Ribbit is active and not yet acknowledged.

**Flow of events (Main Success Scenario):**

1. User views the Pond or notifications.
2. System displays any active or unread reminders/ribbits.
3. User selects a Ribbit.
4. System verifies that the Ribbit belongs to the user's Pond/Relationship and was sent to the user.
5. System displays the Ribbit details.
6. User selects Acknowledge.
7. System records the acknowledgment and optional emoji.
8. System updates the Ribbit status to acknowledged.
9. System optionally notifies the sender that the Ribbit was acknowledged.
10. System updates the Pond view to reflect the interaction.

**Alternate flows:**

**A1 — Reminder already acknowledged**

- **A1.1** At Step 2, system detects the reminder/ribbit has already been acknowledged.
- **A1.2** System displays the acknowledged status.
- **A1.3** No further action is required.

**A2 — User dismisses without acknowledging**

- **A2.1** At Step 5, user exits or dismisses the reminder/ribbit without acknowledging.
- **A2.2** System keeps the reminder/ribbit active/unacknowledged.
- **A2.3** Reminder remains available for later interaction.

**A3 — User is not authorized to view Ribbit**

- **A3.1** At Step 4, system detects that the Ribbit does not belong to the user's Pond/Relationship or was not sent to the user.

- **A3.2** System denies access.
- **A3.3** System does not display the Ribbit details.

**Postconditions:**

- Reminder/ribbit is marked as acknowledged (if completed).
- Sender can see that the reminder/ribbit was acknowledged.
- System updates relationship interaction state accordingly.

**Use case:** AnswerDailyDiscussionPrompt

**ID:** UC-PL-05

**Actors:** PondMate/User

**Preconditions:**

- User is logged in.
- User has a connected PondMate.
- User is a member of the Pond/Relationship.
- A daily discussion prompt is available for that Pond/Relationship.

**Flow of events (Main Success Scenario):**

1. User navigates to the Pond or Discussion section.
2. System displays the current daily discussion prompt.
3. User selects the prompt.
4. System displays an input area for the response.
5. User enters a response.
6. User submits the response.
7. System validates that the prompt belongs to the current Pond/Relationship.
8. System validates and saves the response with the DiscussionPromptId and UserAccountId.
9. System makes the response visible only to the PondMate in the same Pond.
10. System updates the discussion status, such as "1/2 responses completed" or "Complete."

**Alternate flows:**

**A1 — User skips the prompt**

- **A1.1** At Step 3, user chooses not to respond.
- **A1.2** System allows the user to return later.
- **A1.3** Prompt remains active for the day.

**A2 — User edits response**

- **A2.1** After Step 7, user selects to edit their response.



- **A2.2** System allows modification of the existing response.
- **A2.3** System saves the updated response.

### **A3 — Both users have responded**

- **A3.1** At Step 9, system detects both PondMates have submitted responses.
- **A3.2** System marks the discussion as complete.
- **A3.3** System may highlight or surface the shared responses for both users to view.

#### **Postconditions:**

- User's response to the daily discussion prompt is saved.
- PondMate can view the response.
- System tracks discussion completion status for both users.

**Use case:** RequestSmartConnectionAssistantSuggestion

**ID:** UC-PL-06

**Actors:** PondMate/User

#### **Preconditions:**

- User is logged in.
- User has a connected PondMate.
- User is a member of the Pond/Relationship.
- Relevant context exists, such as at least one recent check-in, Ribbit, or discussion activity.

#### **Flow of events (Main Success Scenario):**

1. User selects the Smart Connection Assistant feature.
2. System gathers the minimum relevant context needed from the current Pond/Relationship:
  - User's current mood
  - PondMate's current mood, if available
  - Recent check-in status
  - Active/unanswered Ribbits
  - Recent discussion activity
3. System excludes unnecessary private data and only prepares context needed for the requested suggestion.
4. System sends the prepared context to the AI service.
5. AI generates suggestions:
  - One care action
  - One message suggestion
  - One optional discussion prompt
6. System receives the AI response.
7. System displays the suggestions to the user.

8. User reviews the suggestions.
9. User may:
  - Accept and use a suggestion, or
  - Ignore the suggestions, or
  - Request regeneration
10. System records the interaction (optional, for future context improvement).

**Alternate flows:**

**A1 — Insufficient context**

- **A1.1** At Step 2, system detects limited or missing context.
- **A1.2** System proceeds with minimal available data.
- **A1.3** AI generates more general suggestions.
- **A1.4** Use case continues at Step 4.

**A2 — AI request fails**

- **A2.1** At Step 3, AI service is unavailable or request fails.
- **A2.2** System displays an error message.
- **A2.3** System allows the user to retry.

**A3 — User regenerates suggestions**

- **A3.1** At Step 8, user selects regenerate.
- **A3.2** System resends context to the AI service.
- **A3.3** AI returns new suggestions.
- **A3.4** System displays updated suggestions.

**Postconditions:**

- User receives optional AI-generated suggestions for connection.
- No actions are forced; suggestions remain user-controlled.
- Suggestions remain private to the requesting user unless the user chooses to act on them.
- System may store limited interaction data to improve future suggestions.

**Use case: CreateOrJoinPond**

**ID:** UC-PL-00

**Actors:** PondMate/User

**Preconditions:**

- User is logged in.

- User does not already have an active Pond, or the system allows the user to join/create a Pond.
- User wants to create or join a private relationship space with a PondMate.

**Flow of events (Main Success Scenario):**

1. User enters the Pond page or selects Create/Join Pond.
2. System checks whether the user is already connected to a Pond.
3. System detects that the user is not currently connected to a Pond.
4. System displays options to create a new Pond or join an existing Pond.
5. User chooses to create a new Pond.
6. System creates a new Relationship record.
7. System creates a RelationshipMember record linking the user to the Relationship.
8. System displays the new Pond space.
9. System provides an option to invite or connect a PondMate.

**Alternate flows:**

**A1 — User joins an existing Pond**

- **A1.1** At Step 4, user chooses to join an existing Pond.
- **A1.2** User enters an invite code, link, or other connection information.
- **A1.3** System validates the invitation.
- **A1.4** System creates a RelationshipMember record linking the user to the existing Relationship.
- **A1.5** System displays the shared Pond space.

**A2 — User already has an active Pond**

- **A2.1** At Step 2, system detects that the user is already connected to a Pond.
- **A2.2** System retrieves the existing Pond.
- **A2.3** System displays the existing Pond instead of creating a new one.

**A3 — Invalid or expired invitation**

- **A3.1** At Step A1.3, system detects that the invite code/link is invalid or expired.
- **A3.2** System displays an error message.
- **A3.3** User may retry or choose to create a new Pond.

**Postconditions:**

- User is connected to a private Pond/Relationship.
- A RelationshipMember record exists for the user.
- User can access Pond features associated with that Relationship.